SGX 메모리 관리 및 보안 메커니즘의 실제 구현 분석

개요

이 문서는 Intel SGX SDK의 메모리 관리 및 보안 메커니즘을 실제 코드 레벨에서 분석한 결과를 정리합니다. EPC 메모리 할당, 스택 보호, 힙 관리, 그리고 메모리 암호화 과정의 구체적인 구현을 추적하여 SGX의 메모리 보안 아키텍처를 완전히 이해하는 것이 목표입니다.

1. EPC 메모리 할당 과정의 페이지 테이블 설정 및 권한 관리

1.1 VRD (Virtual Range Descriptor) 시스템

SGX는 EPC 메모리 관리를 위해 VRD(Virtual Range Descriptor) 시스템을 사용합니다.

VRD 구조체 정의 (`sdk/tmm_rsrv/mm_vrd.h`)

```
typedef struct vrd_t {
size_t start_addr; // 시작 주소
size_t size; // 크기 (바이트)
uint32_t flags; // 사용 제한 플래그
uint32_t state; // VRD 상태
uint32_t perms; // 페이지 권한
uint32_t page_type; // 페이지 타입
vrd_t* next; // 다음 VRD 포인터
```

```
vrd_t* prev; // 이전 VRD 포인터

} vrd_t;
```

VRD 상태 정의

```
#define VRD_STATE_FREE 0 // 자유 리스트에 있음

#define VRD_STATE_RESERVED 1 // 예약된 메모리

#define VRD_STATE_RESERVED_SYSTEM 2 // 시스템 예약 메모리

#define VRD_STATE_COMMITTED 3 // 커밋된 메모리

#define VRD_STATE_COMMITTED_SYSTEM 4 // 시스템 커밋 메모리

#define VRD_STATE_LOCKED 5 // 잠긴 메모리
```

페이지 권한 정의

```
#define SGX_PAGE_NOACCESS 0x01 // 접근 불가 (-)

#define SGX_PAGE_READONLY 0x02 // 읽기 전용 (R)

#define SGX_PAGE_READWRITE 0x04 // 읽기/쓰기 (RW)

#define SGX_PAGE_EXECUTE_READ 0x20 // 실행/읽기 (RX)

#define SGX_PAGE_EXECUTE_READWRITE 0x40 // 실행/읽기/쓰기 (RWX)
```

1.2 EPC 메모리 할당 과정 (`sdk/tmm_rsrv/sgx_rsrv_mem.cpp`)

sgx_alloc_rsrv_mem_ex() 함수 분석

```
void * sgx_alloc_rsrv_mem_ex(void *desired_addr, size_t length)
```

```
{
static volatile bool g_first_alloc = true;
vrd_t * rsrv_vrd = NULL;
sgx_status_t sgx_ret = SGX_ERROR_UNEXPECTED;
// 예약 메모리가 설정되지 않은 경우 할당 거부
if (0 == rsrv_mem_size) {
errno = EPERM;
return NULL;
}
// 매개변수 검증: 페이지 정렬, 크기 검증
if(!IS_PAGE_ALIGNED(desired_addr) || length == 0 ||
!IS_PAGE_ALIGNED(length) || length > rsrv_mem_size) {
errno = EINVAL;
return NULL;
}
// 주소 범위 검증
if(desired_addr != NULL) {
size_t end = (size_t)desired_addr + length - 1;
if((size_t)desired_addr > end || length > end ||
end > (size_t)rsrv_mem_base + rsrv_mem_size - 1 ||
(size_t)desired_addr < (size_t)rsrv_mem_base) {
errno = EINVAL;
return NULL;
```

```
}

}


sgx_thread_mutex_lock(&g_vrdl_mutex);

// 첫 번째 할당 시 VRD 시스템 초기화

if(unlikely(g_first_alloc)) {

int ret = init_rsrv_mem_vrd();

if(ret != 0) {

sgx_thread_mutex_unlock(&g_vrdl_mutex);

errno = ret;

return NULL;

}

g_first_alloc = false;

}


// VRD 삽입 및 메모리 할당

rsrv_vrd = insert_vrd((size_t)desired_addr, length,

(!EDMM_supported && g_global_data.rsrv_executable) ?

SGX_PAGE_EXECUTE_READWRITE : SGX_PAGE_READWRITE,

VRD_STATE_COMMITTED,

VRD_PT_REG, &sgx_ret);

if (NULL == rsrv_vrd) {

sgx_thread_mutex_unlock(&g_vrdl_mutex);
```

```
errno = EINVAL;

if(sgx_ret == SGX_ERROR_OUT_OF_MEMORY)

errno = ENOMEM;

return NULL;

}

// EDMM 지원 시 페이지 커밋 처리

if(rsrv_vrd->start_addr + length > (size_t)rsrv_mem_base + rsrv_mem_committed) {

size_t prev_rsrv_mem_committed = rsrv_mem_committed;

void * start_addr = NULL;

size_t size = 0;

size_t offset = (rsrv_vrd->start_addr + length) -

((size_t)rsrv_mem_base + rsrv_mem_committed);

rsrv_mem_committed += ROUND_TO(offset, SE_PAGE_SIZE);

if(EDMM_supported && rsrv_vrd->start_addr + length >

(size_t)rsrv_mem_base + rsrv_mem_min_size) {

// 새 페이지 적용 필요

if(prev_rsrv_mem_committed > rsrv_mem_min_size) {

start_addr = (void *)((size_t)rsrv_mem_base + prev_rsrv_mem_committed);

size = ROUND_TO(offset, SE_PAGE_SIZE);

} else {

start_addr = (void *)((size_t)rsrv_mem_base + rsrv_mem_min_size);

size = rsrv_mem_committed - rsrv_mem_min_size;

}
```

```
// EACCEPT로 새 페이지 수락

int ret = mm_commit(start_addr, size);

if(ret != 0) {

rsrv_mem_committed = prev_rsrv_mem_committed;

remove_vrd(rsrv_vrd);

sgx_thread_mutex_unlock(&g_vrdl_mutex);

errno = ENOMEM;

return NULL;

}

}

}

g_peak_rsrv_mem_committed = g_peak_rsrv_mem_committed < rsrv_mem_committed ?

rsrv_mem_committed : g_peak_rsrv_mem_committed;



void* start_addr = (void *) rsrv_vrd->start_addr;

combine_vrds((size_t)rsrv_vrd->start_addr, (size_t)rsrv_vrd->start_addr + length

sgx_thread_mutex_unlock(&g_vrdl_mutex);

return (void *)start_addr;

}
```

1.3 페이지 권한 변경 메커니즘

sgx_tprotect_rsrv_mem() 함수 분석

```
sgx_status_t sgx_tprotect_rsrv_mem(void *addr, size_t len, int prot)
{
sgx_status_t ret = SGX_ERROR_UNEXPECTED;
// EDMM이 지원되지 않는 경우 제한적 권한 변경만 허용
if(!EDMM_supported) {
uint32_t cur_prot=0;
get_vrds_perms((size_t)addr, &cur_prot);
// 현재 권한이 NOACCESS이거나 RW에서 실행 권한 추가 시도 시 거부
if(SGX_PAGE_NOACCESS == cur_prot ||
(SGX_PAGE_READWRITE == cur_prot && (prot & SGX_PROT_EXEC))) {
return SGX_ERROR_INVALID_PARAMETER;
}
return SGX_SUCCESS;
}
// 매개변수 검증
if(!sgx_is_within_enclave(addr, len))
return SGX_ERROR_INVALID_PARAMETER;
if(0 == rsrv_mem_size || (size_t)addr < (size_t)rsrv_mem_base ||
!IS_PAGE_ALIGNED(addr) || len < SE_PAGE_SIZE || !IS_PAGE_ALIGNED(len)) {
return SGX_ERROR_INVALID_PARAMETER;
}
// 권한 플래그 검증 및 변환
```

```
if((!(prot & SGX_PROT_READ) && (prot & (SGX_PROT_WRITE | SGX_PROT_EXEC))) ||

(prot & ~(SGX_PROT_READ | SGX_PROT_WRITE | SGX_PROT_EXEC))) {

return SGX_ERROR_INVALID_PARAMETER;

}

uint64_t perms = SI_FLAG_NONE;

if((prot & SGX_PROT_EXEC) == SGX_PROT_EXEC) perms |= SI_FLAG_X;

if((prot & SGX_PROT_READ) == SGX_PROT_READ) perms |= SI_FLAG_R;

if((prot & SGX_PROT_WRITE) == SGX_PROT_WRITE) perms |= SI_FLAG_W;



uint32_t protect = convert_si_flags_to_protect(perms);

size_t end_addr = (size_t)addr + len - 1;

sgx_thread_mutex_lock(&g_vrdl_mutex);

// VRD 분할 및 권한 변경

bool brc = split_vrds_if_needed((size_t)addr, end_addr);

if(brc == true) {

vrd_t *vrd = find_vrd((size_t)addr);

if(NULL == vrd) {

ret = SGX_ERROR_UNEXPECTED;

goto cleanup;

}

} else {

ret = SGX_ERROR_OUT_OF_MEMORY;

goto cleanup;
```

```
}

// 페이지 타입 검증

if (!check_vrds_pagetype((size_t)addr, end_addr, VRD_PT_REG)) {

ret = SGX_ERROR_INVALID_PARAMETER;

goto cleanup;

}



// 실제 권한 변경 수행

ret = tprotect_internal((size_t)addr, len, perms);

if(ret != SGX_SUCCESS) {

goto cleanup;

}



// VRD에 새 권한 설정

size_t tmp_addr = (size_t)addr;

while (tmp_addr < end_addr) {

vrd_t *vrd = find_vrd((size_t)tmp_addr);

if (!vrd) {

ret = SGX_ERROR_UNEXPECTED;

goto cleanup;

}

vrd->perms = protect;

tmp_addr = vrd->start_addr + vrd->size;
```

```
}

combine_vrds((size_t)addr, end_addr);
ret = SGX_SUCCESS;

cleanup:
sgx_thread_mutex_unlock(&g_vrdl_mutex);
return ret;
}
```

2. 스택 보호 메커니즘의 Stack Canary 및 가드 페이지 설정

2.1 Stack Canary 구현 (`sdk/compiler-rt/stack_chk.c`)

Stack Canary 검증 함수

```
extern void * __intel_security_cookie;

#ifdef __x86_64__
void __intel_security_check_cookie(void * cookie)
#else
void __attribute__ ((regparm(1))) __intel_security_check_cookie(void * cookie)
#endif
{
```

```
if (cookie == __intel_security_cookie)

return;

abort(); // 스택 오버플로우 감지 시 프로그램 종료

}


void __attribute__((noreturn)) __stack_chk_fail(void)

{

abort(); // GCC 스택 보호 실패 시 호출

}
```

TRTS에서의 Stack Canary 검증 (`sdk/trts/trts.cpp`)

```
extern uintptr_t __stack_chk_guard;


int check_static_stack_canary(void *tcs)

{

size_t *canary = TCS2CANARY(tcs);

if (*canary != (size_t)__stack_chk_guard) {

return -1; // 스택 카나리 손상 감지

}

return 0;

}
```

2.2 외부 스택 할당 및 프로빙 (`sdk/trts/trts.cpp`)

sgx_ocalloc() 함수의 스택 프로빙 메커니즘

```
#define OC_ROUND 16

void * sgx_ocalloc(size_t size)
{
// 현재 SSA에서 외부 스택 주소 읽기
thread_data_t *thread_data = get_thread_data();
ssa_gpr_t *ssa_gpr = reinterpret_cast<ssa_gpr_t *>(thread_data->first_ssa_gpr);
size_t addr = ssa_gpr->REG(sp_u);

// u_rsp가 신뢰할 수 없는 주소를 가리키는지 확인
if(!sgx_is_outside_enclave(reinterpret_cast<void *>(addr), sizeof(size_t))) {
abort();
}

// 할당할 크기가 너무 큰 경우
if(addr < size) {
abort();
}

// 할당된 메모리의 시작 주소 계산
addr -= size;
```

```
addr &= ~(static_cast<size_t>(OC_ROUND - 1)); // 스택 정렬


// 할당된 메모리가 enclave와 겹치는지 확인
if(!sgx_is_outside_enclave(reinterpret_cast<void *>(addr), size)) {
abort();
}


// 외부 스택 프로빙으로 가드 페이지 건드리지 않도록 보장
size_t first_page = TRIM_TO_PAGE(ssa_gpr->REG(sp_u) - 1);
size_t last_page = TRIM_TO_PAGE(addr);


// 무한 루프 방지를 위한 검사
if (last_page == 0) {
abort();
}


// 컴파일러 최적화 방지를 위해 volatile 사용
// 높은 주소에서 낮은 주소 순으로 프로빙
for(volatile size_t page = first_page; page >= last_page; page -= SE_PAGE_SIZE)
// 페이지 폴트 주소가 RSP보다 작을 때 OS가 물리 페이지 커밋을 거부할 수 있으므로
// 페이지 프로빙 전에 외부 스택 주소 업데이트
ssa_gpr->REG(sp_u) = page;
// 실제 페이지 프로빙 수행
```

```
*reinterpret_cast<uint8_t *>(page) = 0;

}


// SSA의 외부 스택 주소를 할당된 주소로 업데이트

ssa_gpr->REG(sp_u) = addr;


return reinterpret_cast<void *>(addr);

}
```

2.3 스택 주소 검증 (sdk/trts/trts_util.cpp)

스택 주소 유효성 검사

```
bool is_stack_addr(void *address, size_t size)

{

thread_data_t *thread_data = get_thread_data();

size_t stack_base = thread_data->stack_base_addr;

size_t stack_limit = thread_data->stack_limit_addr;

size_t addr = (size_t) address;

return (addr <= (addr + size)) &&

(stack_base >= (addr + size)) &&

(stack_limit <= addr);

}
```

```
bool is_valid_sp(uintptr_t sp)

{

return (!(sp & (sizeof(uintptr_t) - 1)) && // sp는 4/8바이트 정렬되어야 함

is_stack_addr((void*)sp, 0)); // sp는 스택 영역을 가리켜야 함

}
```

3. 힙 관리 시스템의 동적 메모리 할당 및 해제

3.1 C++ 메모리 관리 래퍼 (`sdk/cpprt/memory_manage/`)

operator new 구현 (`sdk/cpprt/memory_manage/new1.cpp`)

```
SGX_WEAK void* SGXAPI operator new(size_t dwBytes)

{

void* address = malloc(dwBytes);

while (address == NULL) {

if (!call_newh()) {

throw std::bad_alloc();

}

address = malloc(dwBytes);

}

return address;

}
```

operator delete 구현 (`sdk/cpprt/memory_manage/delete1.cpp`)

```
SGX_WEAK void SGXAPI operator delete (void* ptr)
{
free(ptr);
}
```

new_handler 메커니즘 (`sdk/cpprt/memory_manage/new_handler.cpp`)

```
namespace std {
static sgx_spinlock_t handler_lock = SGX_SPINLOCK_INITIALIZER;
static new_handler new_handl = NULL;

new_handler set_new_handler(new_handler handle)
{
sgx_spin_lock(&handler_lock);
new_handler retHandle = new_handl;
if (handle == NULL) {
new_handl = NULL;
} else if (sgx_is_within_enclave((void *)handle, 0)) {
// 핸들러 주소가 enclave 내부에 있을 때만 설정
new_handl = handle;
}
sgx_spin_unlock(&handler_lock);
return retHandle;
```

```
}
}

int call_newh()
{
int ret = 0;
sgx_spin_lock(&std::handler_lock);
std::new_handler handler = std::new_handl;
// 데드락 방지를 위해 핸들러 호출 전 언락
sgx_spin_unlock(&std::handler_lock);

if (handler != NULL) {
handler();
ret = 1;
}
return ret;
}
```

3.2 dlmalloc 기반 힙 구현 (`sdk/tlibc/stdlib/malloc.c`)

SGX는 Doug Lea의 dlmalloc 2.8.6을 기반으로 한 힙 관리자를 사용합니다.

주요 설정

```
#ifdef _TLIBC_
#define HAVE_MMAP 0 // mmap 사용 안 함
#define HAVE_MREMAP 0 // mremap 사용 안 함
#define HAVE_MORECORE 1 // sbrk 스타일 할당 사용
#define MMAP_CLEARS 0 // mmap이 메모리를 지우지 않음
#define MALLOC_ALIGNMENT ((size_t)16U) // 16바이트 정렬
#define USE_LOCKS 1 // 락 사용
#define USE_SPIN_LOCKS 1 // 스핀락 사용
#define FOOTERS 1 // 보안을 위한 푸터 사용
#define REALLOC_ZERO_BYTES_FREES 1 // realloc(p, 0)이 free(p)와 동일
#endif
```

보안 기능

- **FOOTERS**: 각 할당된 청크가 추가 검사 워드를 가져 악의적인 가짜 청크 해제 방지
- **USE_LOCKS**: 모든 public 함수 호출을 락으로 보호
- **경계 검사**: 힙 베이스 아래 메모리 위치 수정 방지
- **이중 해제 감지**: 부적절한 free 및 realloc 감지

3.3 힙 초기화 및 관리 (`sdk/trts/trts_util.cpp`)

힙 정보 조회 함수들

```
void * get_heap_base(void)
{
```

```
return GET_PTR(void, get_enclave_base(), g_global_data.heap_offset);

}



size_t get_heap_size(void)

{

size_t heap_size = g_global_data.heap_size;

if (EDMM_supported) {

for(uint32_t i = 0; i < g_global_data.layout_entry_num; i++) {

if(g_global_data.layout_table[i].entry.id == LAYOUT_ID_HEAP_MAX) {

heap_size += ((size_t)g_global_data.layout_table[i].entry.page_count << SE_PAGE_

}

}

}

return heap_size;

}



size_t get_heap_min_size(void)

{

size_t heap_size = 0;

for(uint32_t i = 0; i < g_global_data.layout_entry_num; i++) {

if(g_global_data.layout_table[i].entry.id == LAYOUT_ID_HEAP_MIN) {

heap_size = ((size_t)g_global_data.layout_table[i].entry.page_count << SE_PAGE_S

break;
```

```
}

}

return heap_size;

}
```

4. 메모리 암호화 과정의 MEE(Memory Encryption Engine) 상호작용

4.1 메모리 경계 검사 (`sdk/trts/trts.cpp`)

Enclave 내부/외부 메모리 검증

```
int sgx_is_within_enclave(const void *addr, size_t size)

{

size_t start = reinterpret_cast<size_t>(addr);

size_t end = 0;

size_t enclave_start = (size_t)g_enclave_base;

size_t enclave_end = enclave_start + (size_t)g_enclave_size - 1;



if(size > 0) {

end = start + size - 1;

} else {

end = start;

}

if((start <= end) && (start >= enclave_start) && (end <= enclave_end)) {
```

```
return 1;
}
return 0;
}

int sgx_is_outside_enclave(const void *addr, size_t size)
{
size_t start = reinterpret_cast<size_t>(addr);
size_t end = 0;
size_t enclave_start = (size_t)g_enclave_base;
size_t enclave_end = enclave_start + (size_t)g_enclave_size - 1;

if(size > 0) {
end = start + size - 1;
} else {
end = start;
}
if((start <= end) && ((end < enclave_start) || (start > enclave_end))) {
return 1;
}
return 0;
}
```

4.2 EPC 페이지 생성 및 권한 설정 (`psw/urts/loader.cpp`)

메모리 영역 구축 과정

```
int CLoader::build_mem_region(const section_info_t &sec_info)
{
int ret = SGX_SUCCESS;
uint64_t offset = 0;
sec_info_t sinfo;
memset(&sinfo, 0, sizeof(sinfo));

// 초기화된 데이터를 포함하는 섹션의 페이지들을 구축
// 각 페이지는 재배치 데이터를 포함할 수 있으므로 개별적으로 추가되어야 함
while(offset < sec_info.raw_data_size) {
uint64_t rva = sec_info.rva + offset;
uint64_t size = MIN((SE_PAGE_SIZE - PAGE_OFFSET(rva)),
(sec_info.raw_data_size - offset));
sinfo.flags = sec_info.flag;

// 재배치 페이지이고 쓰기 권한이 없는 경우 쓰기 권한 추가
if(is_relocation_page(rva, sec_info.bitmap) && !(sec_info.flag & SI_FLAG_W)) {
sinfo.flags = sec_info.flag | SI_FLAG_W;
assert(g_enclave_creator != NULL);
if(g_enclave_creator->use_se_hw() == true) {
```

```
ret = mprotect((void*)(TRIM_TO_PAGE(rva) + (uint64_t)m_start_addr),

SE_PAGE_SIZE,

(int)(sinfo.flags & SI_MASK_MEM_ATTRIBUTE));

if(ret != 0) {

SE_TRACE(SE_TRACE_WARNING,

"mprotect(rva=0x%llx, len=%d, flags=%d) failed\n",

rva, SE_PAGE_SIZE, int(sinfo.flags & SI_MASK_MEM_ATTRIBUTE));

return SGX_ERROR_UNEXPECTED;

}

}

}



if (size == SE_PAGE_SIZE)

ret = build_pages(rva, size, sec_info.raw_data + offset, sinfo, ADD_EXTEND_PAGE)

else

ret = build_partial_page(rva, size, sec_info.raw_data + offset, sinfo, ADD_EXTEN

if(SGX_SUCCESS != ret) {

return ret;

}

offset += size;

}

return SGX_SUCCESS;
```

```
}
```

4.3 난수 생성 및 보안 (sdk/trts/trts.cpp)

하드웨어 난수 생성기 사용

```
sgx_status_t sgx_read_rand(unsigned char *rand, size_t length_in_bytes)
{
// 매개변수 검증
if(!rand || !length_in_bytes) {
return SGX_ERROR_INVALID_PARAMETER;
}
if(!sgx_is_within_enclave(rand, length_in_bytes) &&
!sgx_is_outside_enclave(rand, length_in_bytes)) {
return SGX_ERROR_INVALID_PARAMETER;
}
// RDRAND를 사용한 난수 생성 루프
uint32_t rand_num = 0;
while(length_in_bytes > 0) {
sgx_status_t status = __do_get_rand32(&rand_num);
if(status != SGX_SUCCESS) {
return status;
}
```

```
size_t size = (length_in_bytes < sizeof(rand_num)) ?

length_in_bytes : sizeof(rand_num);

memcpy(rand, &rand_num, size);



rand += size;

length_in_bytes -= size;

}

// 보안을 위해 임시 난수 값 지우기

memset_s(&rand_num, sizeof(rand_num), 0, sizeof(rand_num));

return SGX_SUCCESS;

}



static sgx_status_t __do_get_rand32(uint32_t* rand_num)

{

#ifndef SE_SIM

// 하드웨어 모드에서는 CPU가 RDRAND를 지원한다고 가정

// do_rdrand()는 RDRAND를 10번 시도

if(0 == do_rdrand(rand_num))

return SGX_ERROR_UNEXPECTED;

#else

// 시뮬레이션 모드에서는 CPU가 RDRAND를 지원하면 사용, 아니면 LCG 사용

if(TEST_CPU_HAS_RDRAND) {

if(0 == do_rdrand(rand_num))
```

```
return SGX_ERROR_UNEXPECTED;

} else {

*rand_num = get_rand_lcg();

}

#endif

return SGX_SUCCESS;

}
```

5. 메모리 보안 메커니즘 종합

5.1 다층 보안 아키텍처

1. **하드웨어 레벨**: MEE(Memory Encryption Engine)가 EPC 메모리 암호화
2. **페이지 레벨**: VRD 시스템을 통한 페이지 권한 및 상태 관리
3. **스택 레벨**: Stack Canary와 가드 페이지를 통한 스택 오버플로우 방지
4. **힙 레벨**: dlmalloc의 보안 기능과 경계 검사
5. **애플리케이션 레벨**: 메모리 경계 검증 함수들

5.2 메모리 격리 메커니즘

- **Enclave 경계 검사**: `sgx_is_within_enclave()`, `sgx_is_outside_enclave()`
- **스택 격리**: 외부 스택과 내부 스택 분리, 프로빙을 통한 가드 페이지 보호
- **힙 격리**: Enclave 내부 힙과 외부 메모리 완전 분리
- **페이지 권한 관리**: VRD를 통한 세밀한 페이지 권한 제어

5.3 공격 방어 메커니즘

- **버퍼 오버플로우**: Stack Canary, 가드 페이지, 힙 경계 검사
- **Use-after-free**: dlmalloc의 이중 해제 감지
- **메모리 손상**: FOOTERS를 통한 힙 무결성 검증
- **사이드 채널**: 메모리 접근 패턴 은닉, 일정한 실행 시간 보장

결론

Intel SGX의 메모리 관리 및 보안 메커니즘은 하드웨어부터 애플리케이션 레벨까지 다층적인 보안 아키텍처를 구현합니다. VRD 시스템을 통한 세밀한 메모리 관리, Stack Canary와 가드 페이지를 통한 스택 보호, dlmalloc 기반의 안전한 힙 관리, 그리고 MEE를 통한 하드웨어 레벨 암호화가 유기적으로 결합되어 강력한 메모리 보안을 제공합니다.

이러한 메커니즘들은 단순히 개별적으로 동작하는 것이 아니라, 서로 상호작용하며 포괄적인 보안 환경을 구축합니다. 특히 EDMM(Enclave Dynamic Memory Management) 지원을 통해 동적 메모리 관리가 가능하면서도 보안성을 유지하는 것이 SGX 메모리 관리의 핵심 특징입니다.